Exception Handling in C#

1. What is an Exception?

An **exception** is a **runtime error** that occurs while executing a program.

When an exception occurs:

- The **normal flow of the program stops**
- The **CLR (Common Language Runtime)** creates an **exception object**
- The program **searches for a handler** to manage that error

If not handled → **application crashes**.

Example

```
int a = 10;

int b = 0;


int result = a / b; // Runtime exception
```

Output:

System.DivideByZeroException

---

2. Why Exception Handling is Important

Exception handling helps to:

1. Prevent application crashes
2. Provide meaningful error messages
3. Maintain application stability
4. Handle unexpected situations
5. Log errors for debugging

Example scenario:

Banking system → transaction fails → system should **handle error gracefully instead of crashing**.

---

3. Exception Handling Structure in C#

The basic structure:

```
try
{
   // Code that may cause exception
}
catch(Exception ex)
{
   // Handle exception
}
finally
{
   // Cleanup code
}
```

Flow

Try Block

↓

Exception occurs?

↓

Yes → Catch Block

↓

Finally Block executes

---

4. try Block

The **try block contains risky code** that may throw exceptions.

Example

```
try
{
   int x = 10;
   int y = 0;
   int result = x / y;
}
```

If an exception occurs → control goes to **catch block**.

---

5. catch Block

The **catch block handles the exception**.

Syntax

```
catch(ExceptionType variable)
{
}
```

Example:

```
try
{
   int x = 10;
   int y = 0;
   int result = x / y;
}
catch(DivideByZeroException ex)
{
```

```
    Console.WriteLine("Cannot divide by zero");
}
```

Output

Cannot divide by zero

---

6. finally Block

The **finally block always executes**, whether an exception occurs or not.

Used for:

- Closing database connections
- Releasing resources
- File closing

Example:

```
try
{
    Console.WriteLine("Processing...");
}
catch(Exception ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Cleanup completed");
}
```

---

7. Complete Example

```
using System;

class Program
{
    static void Main()
    {
        try
        {
            int num1 = 20;
            int num2 = 0;

            int result = num1 / num2;

            Console.WriteLine(result);
        }
        catch(DivideByZeroException ex)
        {
            Console.WriteLine("Error: Division by zero is not allowed");
        }
        finally
        {
            Console.WriteLine("Program finished execution");
        }
    }
}
```

Output

Error: Division by zero is not allowed

Program finished execution

---

8. Multiple Catch Blocks

You can handle **different exceptions separately**.

Example:

```
try
{
   int[] numbers = {1,2,3};


   Console.WriteLine(numbers[5]);
}
catch(IndexOutOfRangeException ex)
{
   Console.WriteLine("Array index error");
}
catch(Exception ex)
{
   Console.WriteLine("General error");
}
```

Best practice:

- **Specific exception first**
- **Generic exception last**

---

9. Exception Class Hierarchy

All exceptions derive from the base class:

System.Object

↓

System.Exception

↓

System.SystemException

↓

Specific Exceptions

Common exceptions:

Exception	**Description**
DivideByZeroException	Division by zero
NullReferenceException	Object is null
IndexOutOfRangeException	Array index error
FormatException	Invalid format
FileNotFoundException	File missing

Example:

```
string value = null;

Console.WriteLine(value.Length);
```

Throws:

NullReferenceException

---

10. Throwing Exceptions (throw)

You can **manually throw exceptions**.

Example:

```
int age = -5;
```

```
if(age < 0)
{
    throw new Exception("Age cannot be negative");
}
```

Output

Unhandled Exception: Age cannot be negative

---

11. Rethrowing Exceptions

Sometimes you **catch an exception and pass it upward**.

```
try
{
    // risky code
}
catch(Exception ex)
{
    Console.WriteLine("Logging error");
    throw;
}
```

Important:

throw;

preserves the **original stack trace**.

Avoid:

throw ex; ❌

---

12. Custom Exceptions

In real applications, we create **custom exceptions**.

Example:

```
public class InvalidAgeException : Exception
{
    public InvalidAgeException(string message) : base(message)
    {
    }
}
```

Usage:

```
int age = -10;

if(age < 0)
{
    throw new InvalidAgeException("Age cannot be negative");
}
```

---

13. Exception Filters (Modern C#)

Modern C# supports **exception filters** using when.

Example:

```
try
{
    int number = int.Parse("abc");
}
catch(FormatException ex) when(ex.Message.Contains("Input"))
{
    Console.WriteLine("Invalid number format");
}
```

Benefit:

- Avoid unnecessary catch blocks
- More control

---

14. Using Pattern Matching in Catch (Modern C#)

```
catch(Exception ex) when (ex is DivideByZeroException)
{
   Console.WriteLine("Division error");
}
```

---

15. Best Practices (Very Important for Interviews)

1️⃣ Catch specific exceptions

Good

```
catch(FileNotFoundException ex)
```

Bad

```
catch(Exception ex)
```

2️⃣ Do not swallow exceptions

Bad practice:

```
catch(Exception)
{
}
```

3️⃣ Always log exceptions

Example:

```
catch(Exception ex)
```

```
{
    Console.WriteLine(ex.Message);
}
```

In real apps:

Serilog

NLog

ILogger

4️⃣ Use finally or using

For resources:

```
using(FileStream fs = new FileStream("test.txt", FileMode.Open))
{
}
```

---

17. Exception Handling Flow Diagram

Start

↓

Try Block

↓

Exception occurs?

↓

Yes → Catch Block

↓

Finally Block

↓

Program Continues

Quick Summary

Keyword	Purpose
try	risky code
catch	handle exception
finally	cleanup code
throw	manually raise exception
custom exception	user-defined error